AI Listing Generate

目录

- AI Listing Generate
  - 技术栈
  - 1. 项目架构图
  - 2. 主流程图（商品生成）
  - 3. 时序图（异步任务 + 轮询）
  - 4. 目录分层
  - 5. 快速启动
    - 5.1 后端
    - 5.2 前端
    - 5.3 Embedding 向量服务（Docker）
  - 6. 配置说明
  - 7. 已实现的关键优化
  - 8. 测试与报告
  - 9. Docker Compose（一键依赖环境）

AI Listing Generate

这个项目核心是在做一件事：把“电商商品上架文案与类目整理”自动化，减少人工编辑和跨平台适配成本。

解决的问题

1.不同站点类目体系不一致，人工选类目容易错。

2.卖家上架商品时，标题/描述/属性生成很耗时，且质量不稳定。

3.多语言翻译和图片 OCR 是高频辅助动作，手工处理效率低。

4.需要可追踪任务（同步返回或异步回调），便于系统集成。

技术栈

- 后端：FastAPI

- 前端：Vue3 + Vite
- 数据库：MySQL（SQLAlchemy）
- 缓存：Redis（类目缓存、Embedding 缓存）
- 大模型：Qwen（通过兼容 OpenAI 接口调用）
- 向量能力：Embedding API（本地或远端）
- 包与环境：uv / Python 3.11+

1. 项目架构图

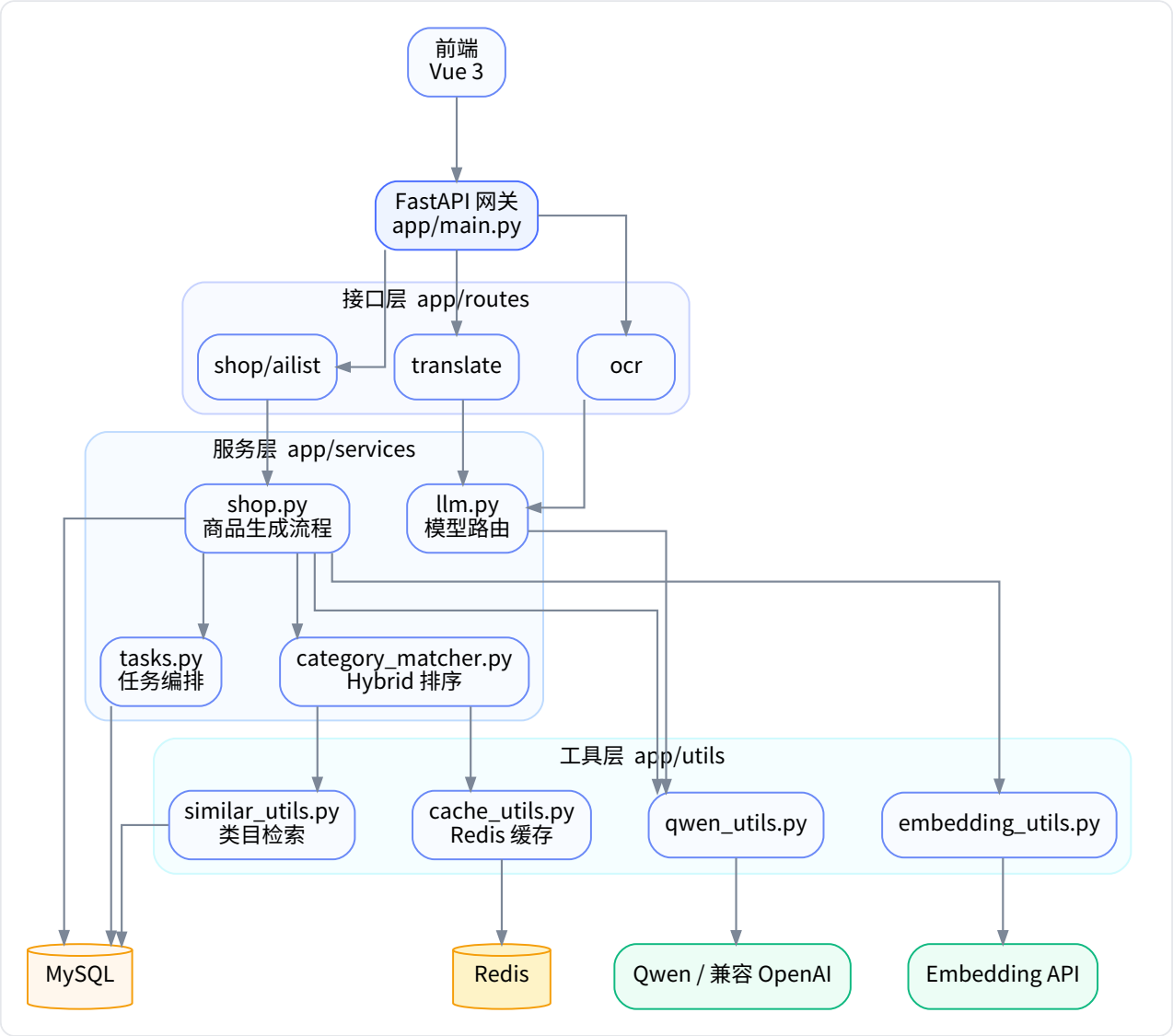

图 1 项目整体架构

2. 主流程图（商品生成）

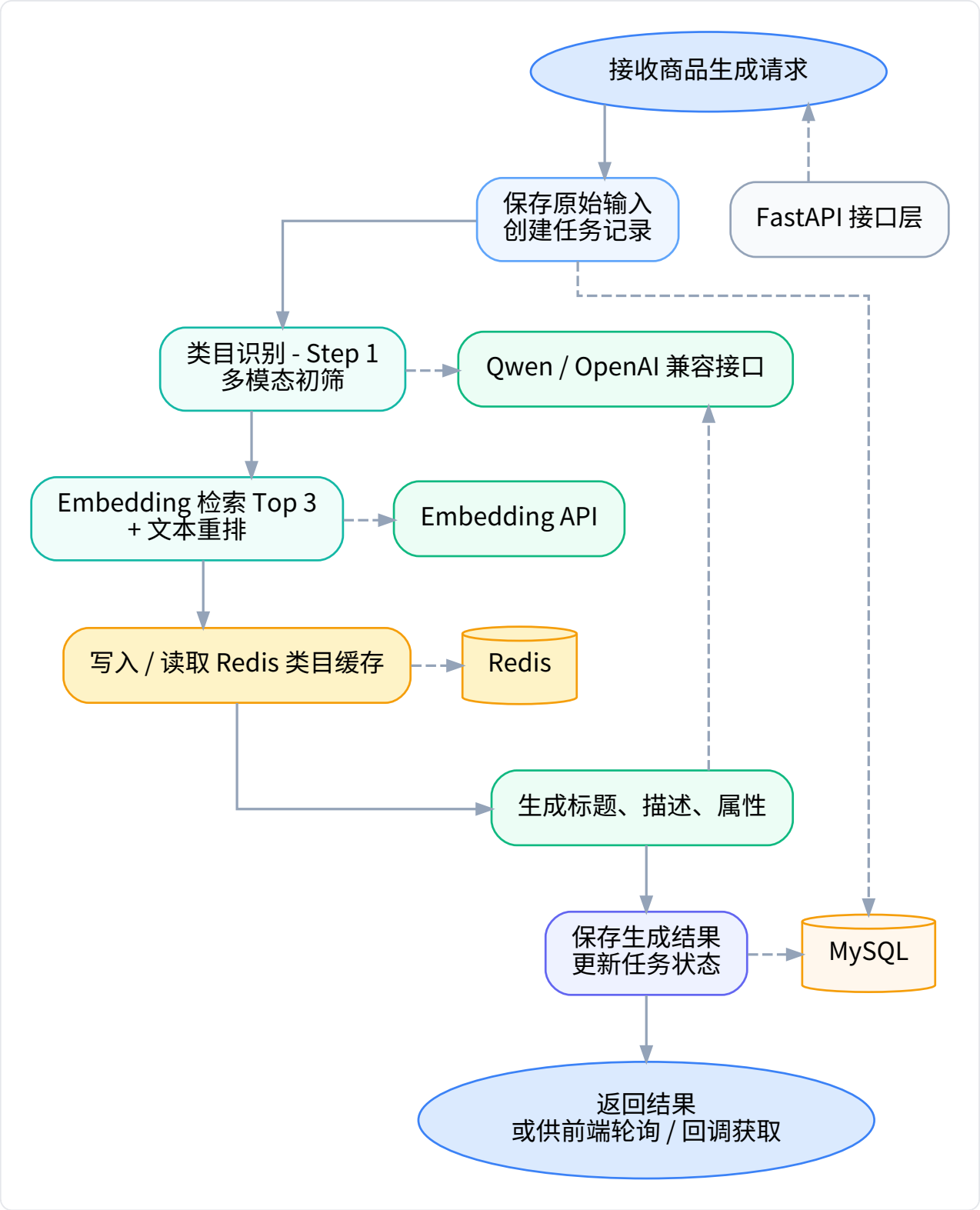

图 2 商品生成主流程

3. 时序图（异步任务 + 轮询）

```
sequenceDiagram
    participant FE as Frontend
    participant API as FastAPI
    participant DB as MySQL
    participant REDIS as Redis
    participant LLM as LLM
    participant EMB as Embedding

    FE->>API: POST /shop/ailist
    API->>DB: Save source and create task
    API-->>FE: task_id and polling_url

    loop Polling
      FE->>API: GET /shop/ailist/task/{task_id}
      API-->>FE: READY/PROCESSING/SUCCESS/FAIL
    end

    API->>REDIS: Read category cache
    alt Cache hit
      REDIS-->>API: final category
    else Cache miss
      API->>LLM: category step1 multimodal
      API->>EMB: hybrid retrieve top3
      API->>LLM: category step2 text rerank
      API->>EMB: hybrid pick top1
      API->>REDIS: write category cache
    end

    API->>LLM: one call title desc attrs
    API->>DB: save output and update task status
```

---

4. 目录分层

- `app/main.py`：应用入口、路由注册、中间件
- `app/routes/`：接口层（参数接收、调用 service、返回响应）
- `app/services/`：业务流程编排（任务、类目、生成）
- `app/utils/`：模型调用、embedding、缓存、通用工具
- `app/models/`：ORM 模型
- `app/schemas/`：请求/响应结构
- `app/database.py`：数据库连接管理
- `app/config.py`：统一配置与日志
- `app/sql/`：建表/初始化 SQL
- `frontend/`：Vue3 + Vite 前端

5. 快速启动

5.1 后端

```
uv venv .venv
uv sync
uv run uvicorn app.main:app --reload
```

5.2 前端

```
cd frontend
npm install
npm run dev
```

5.3 Embedding 向量服务（Docker）

```
docker run --gpus all -p 8000:80 -v "%cd%\data:/data" ghcr.io/huggingface/text-
        embeddings-inference:cuda-1.8.1 --model-id BAAI/bge-m3
```

6. 配置说明

项目支持从根目录 `.env` 读取配置（`app/config.py` 已加载）。

可参考 `.env.example`：

- `MYSQL_USERNAME`
- `MYSQL_PASSWORD`
- `MYSQL_HOST`
- `MYSQL_PORT`
- `MYSQL_DATABASE`
- `REDIS_URL`
- `CATEGORY_CACHE_TTL_SECONDS`
- `EMBEDDING_CACHE_TTL_SECONDS`

7. 已实现的关键优化

- 类目第二步从多模态改为文本重排，降低时延
- 类目结果 Redis 缓存（重复请求明显提速）

- Embedding Redis 缓存（单条与批量均支持）
- 商品文案合并为一次生成（标题+描述+属性）

---

8. 测试与报告

- API 测试目录：`app/test_api/`
- 并发压测报告：`app/test_api/concurrency_test_report_20260407.md`

---

9. Docker Compose（一键依赖环境）

项目根目录已提供：

- `docker-compose.yml`
- `.env.compose.example`

使用步骤：

1. 复制环境文件

```
cp .env.compose.example .env.compose
```

1. 启动依赖（MySQL + Redis + Embedding）

```
docker compose --env-file .env.compose up -d
```

1. 后端 `.env` 建议配置

```
MYSQL_HOST=localhost
MYSQL_PORT=3306
MYSQL_USERNAME=root
MYSQL_PASSWORD=123456
MYSQL_DATABASE=ai_list

REDIS_URL=redis://localhost:6379/0
```

说明：

- 首次启动 MySQL 会自动执行 `app/sql/create_tables.sql` 与 `app/sql/insert_data.sql`。
- 若你已有现成 embedding 镜像，可在 `.env.compose` 中替换 `EMBEDDING_IMAGE`。